

ORIGINAL ARTICLE



Intelligent forecasting of time series based on evolving distributed Neuro-Fuzzy network

Selmo Eduardo Rodrigues Júnior¹ | Ginalber Luiz de Oliveira Serra²

¹Department of Professional Education, Federal Institute of Education, Science and Technology, Campus Imperatriz, Imperatriz, Maranhão, Brazil

²Department of Electrical Engineering, Federal Institute of Education, Science and Technology, Campus Monte Castelo, São Luís, Maranhão, Brazil

Correspondence

Ginalber Luiz de Oliveira Serra, Federal Institute of Education, Science and Technology, São Luís, Maranhão, Brazil.
Email: ginalber@ifma.edu.br

Abstract

An evolving methodology based on Neuro-Fuzzy Takagi-Sugeno network (NF-TS) for distributed forecasting of univariate time series, is proposed. First, the unobservable components, or hidden patterns, are extracted from experimental data of the time series. Then, a distributed forecasting is performed separately for each component, considering an evolving NF-TS associated with each extracted pattern. The evolving NF-TS uses components data to adapt and adjust its structure, as the number of fuzzy rules increases or decreases according the behavior of the unobservable components. A recursive version of singular spectral analysis (SSA) technique is formulated, as one of the main contributions of this article, and it is applied to extract the components. The efficiency of proposed methodology is illustrated from results of comparison to others state-of-the-art techniques for forecasting of various univariate time series.

KEYWORDS

distributed Neuro-Fuzzy Takagi-Sugeno, evolving systems, hidden patterns, time series forecasting, unobservable components

1 | INTRODUCTION

In virtue of technological advance, the amount of available data from various sources and formats has been growing exponentially. A specific category of data is organized sequentially in time, whose denomination is time series. A time series is a sequence of numerical data points in successive and temporal order.¹ The time series exist in the most diverse knowledge areas



and applications, such as finances, social sciences, medicine, meteorology, signal processing, and so on. An important task is the time series forecasting, which consists of projecting the future behavior of data based on current and historical samples.²

Forecasting results are interesting for experts to support the decision-making process in various situations. Many approaches have been developed to perform the time series forecasting. In De Gooijer and Hyndman,¹ a review of evolution over the years in time series forecasting area is presented. In this article, the authors highlight a set of researches that use classic methods based on exponential smoothing, autoregressive integrated moving average (ARIMA), models in state spaces, Box-Jenkins, ARCH-GARCH models, and so on. However, because of the complexity, nonlinearity, and nonstationary behavior of real time series, computational intelligence, and machine learning techniques are considered to time series forecasting task.

There are many articles that considers neural network applications for time series forecasting.³ To predict the price indices of stock markets, Wang et al⁴ developed an architecture which combined Elman recurrent neural networks (ERNN) with stochastic time effective function, by analyzing the proposed model with the linear regression, complexity invariant distance (CID) and multiscale CID analysis methods, and taking the model compared with different models such as the backpropagation neural network, the stochastic time effective neural network, and the ERNN. In Wang et al,⁵ a hybrid model which is distinctive in integrating the advantages of ARIMA and artificial neural networks (ANNs) in modeling the linear and nonlinear behaviors in the dataset, is proposed. The hybrid model was tested on three sets of actual data, namely, the Wolf's sunspot data, the Canadian lynx data and the IBM stock price data. In Yolcu et al,⁶ a novel neural network model, which consists of both linear and nonlinear structures, is proposed. It considers that time series has both linear and nonlinear components, the multiplicative and Mc Culloch-Pitts neuron structures are employed for nonlinear and linear parts of the proposed model, respectively, and the modified particle swarm optimization method is used to train the proposed neural network model.

These described approaches are hybrid, that is, they use a combination of different methods in order to take advantage of each one for time series forecasting. According to Tealab et al,⁷ the common neural networks are not efficient to recognize the behavior of nonlinear or dynamic time series that have moving average characteristics, therefore, they can present poor forecasting. Thus, the authors suggest the formulation of new models of neural networks, such as Deep Learning or hybrid methodologies, whose approaches have been widely explored currently. In Ni et al,⁸ the authors proposed a C-RNN forecasting method for Forex time series data based on deep-recurrent neural network (RNN) and deep convolutional neural network, which can further improve the prediction accuracy of deep learning algorithm for the time series data of exchange rate. Another forecasting method proposed by Yang and Chen⁹ is a hybrid deep learning and empirical mode decomposition model for multistep ahead forecasting to predict the traffic flow and random time series, where the time series decomposition into a collection of patterns and residuals can improve the forecasting performed by neural networks.

The decomposition of time series into simpler components has been considered in many articles, obtaining promising results in forecasting. In Joo and Kim,¹⁰ the authors developed an approach that decomposes the original time series into the trend and variation parts and constructs a separate model for each part, based on wavelet filtering. Another research that follows this reasoning is in Nguyen and Novak.¹¹ In this case, the time series is decomposed into cyclical, trend, seasonal components, and an irregular component. The forecasting result is the combination of the individual forecasting of each component. Trend, cyclic, and seasonal components are represented by models based on fuzzy logic, while the irregular component uses a traditional



Box-Jenkins model. In situations where data are generated at a high velocity, are complex and nonlinear, the evolving systems paradigm can be applied. In this paradigm, the developed models can modify their structure, that is, they adapt according to the changes in environment. The evolving models are applied in situations where purely adaptive models are not sufficient to represent the data.¹²⁻¹⁵

Considering this presented scenario, this article proposes a methodology based on a set of evolving Neuro-Fuzzy Takagi-Sugeno (NF-TS) networks applied for recursive multistep and distributed forecasting of univariate time series, considering the unobservable components extracted by a approach based on spectral singular analysis (SSA) method. Each extracted unobservable component has an associated NF-TS to perform its individual forecasting. Then, the forecasts of all components are grouped to result in original time series forecasting. The evolving approach is based on autonomous learning of multimodel systems version one (ALMMo-1) method.¹² Thus, the number of clouds or fuzzy rules can be increased or decreased for each NF-TS. A set of IF-THEN fuzzy rules is used for forecasting the time series.¹⁶ The originality from main contributions of this article corresponds to the distributed structure of evolving NF-TS network for forecasting each unobservable component, that is, each evolving NF-TS is applied for forecasting a specific unobservable component from time series experimental data. In addition to that, a recursive version of SSA for extracting the unobservable components, is proposed.

This article is organized as follows. Section 2 describes the structure and the mathematical formulation of proposed methodology for time series forecasting. It is presented the extraction of unobservable components, the operation of evolving and distributed NF-TS, and the validation process. Section 3 shows the results of this proposed methodology compared with other articles methods. Finally, Section 4 presents the conclusions of this article and future works.

2 | PROPOSED METHODOLOGY FOR DISTRIBUTED FORECASTING

The proposed methodology is represented by Figure 1. The first step of this approach is to extract patterns or unobservable components from time series y_k , for $k = 1, 2, \dots$. In this article, a recursive version of SSA method is applied. The main advantage of SSA is the possibility to extract n unobservable components, where n is determined by experts.¹⁷ Therefore, these components do not have predefined characteristics such as seasonality, trend, and so on. Their behavior depends on the eigenvalues and eigenvectors identified from the historical data of time series. These components or patterns have simpler behavior to forecast.^{10,11} For each extracted unobservable component, a NF-TS model structure is associated to provide its forecasting. For example, if $n = 3$, there will be 3 NF-TS in this approach. Then, these NF-TS has a differential characteristic as distributed, because n NF-TS are used for forecasting each unobservable component.

Note that each NF-TS uses data previously extracted from its associated unobservable component at times $k - 1, k - 2, \dots, k - s$ to forecast instant k . In addition, each NF-TS makes multiple one-step forecasting, so, they are considered recursive multistep forecasting structures. These NF-TS are evolving, that is, they add or remove fuzzy rules in order to adapt online to data for each instant. Finally, taking into account the forecasts of each component by NF-TS, these results are grouped to obtain the forecasts of original time series. The following subsections describe the extraction of unobservable components and the mathematical formulation of distributed NF-TS. It is important to highlight that the univariate time series is decomposed in a multivariable unobservable components which are used for forecasting of the original time series.

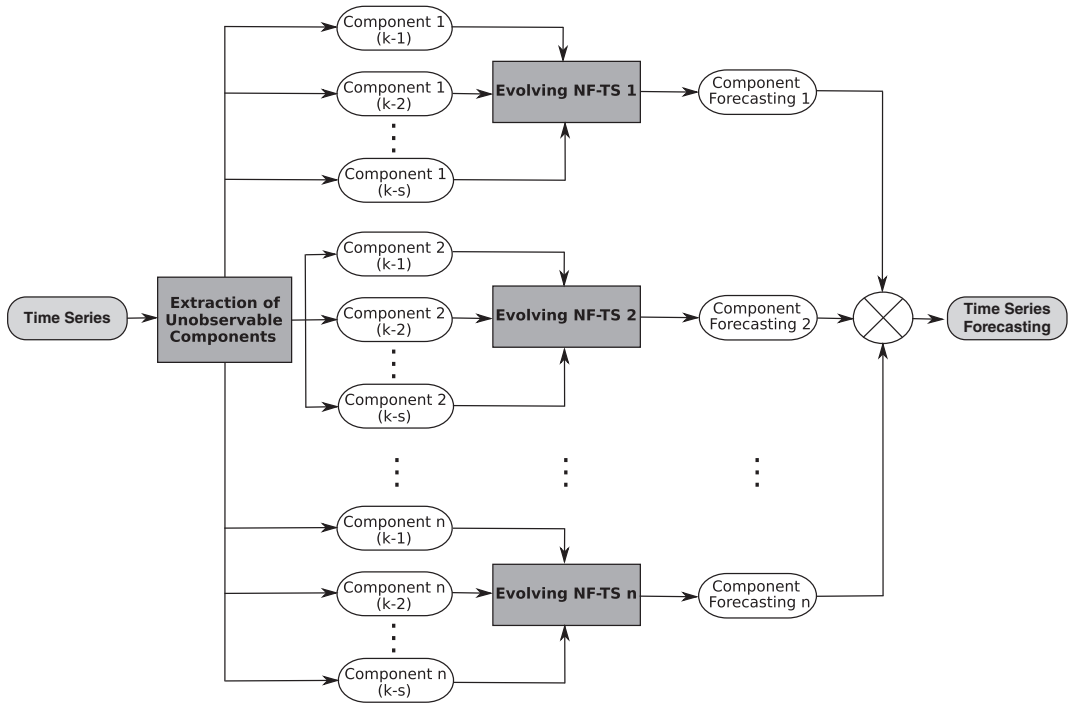


FIGURE 1 Structure of proposed methodology based on evolving and distributed Neuro-Fuzzy Takagi-Sugeno network

2.1 | Unobservable components extraction

Several methods can extract the unobservable components.¹⁷⁻¹⁹ In this article, a recursive version of SSA method is applied to extract n components. This procedure is described in following subsections.

2.1.1 | Initial conditions

First, three parameters must be defined:

- w , length of time series data window to perform the initial conditions procedure;
- l , dimension of covariance matrix;
- n , number of unobservable components extracted from time series.

Let y_k be a sample of univariate time series in instant k . An initial window is created $\{y_k | k = 1, 2, \dots, w\}$, containing the first w observations of time series. Considering this initial window, the trajectory matrix $\mathbf{M}$ represents the time series data in two dimensions as follows:

$$\mathbf{M} = \begin{bmatrix} y_1 & y_2 & \dots & y_{\zeta_w} \\ y_2 & y_3 & \dots & y_{\zeta_w+1} \\ \vdots & \vdots & \vdots & \vdots \\ y_l & y_{l+1} & \dots & y_w \end{bmatrix}, \mathbf{M} \in \mathbb{R}^{l \times \zeta_w}, \quad (1)$$



where $\zeta_w = w - l + 1$. From trajectory matrix, the determination of initial covariance matrix $\mathbf{S}_w$ considering the w first observations (initial window) of time series is performed:

$$\mathbf{S}_w = \mathbf{M}\mathbf{M}^\top, \mathbf{S}_w \in \mathbb{R}^{l \times l}. \quad (2)$$

2.1.2 | Recursive updating of covariance matrix

For each new time series sample y_k , the initial covariance matrix $\mathbf{S}_w$ is updated recursively in $k = w + 1, w + 2, \dots$, and so on. The ζ_k parameter must be incremented by one: $\zeta_k = \zeta_{k-1} + 1$. Next, the updating vector $\mathbf{q}_k$ is formulated as:

$$\mathbf{q}_k = \begin{bmatrix} y_{\zeta_k} & y_{\zeta_k+1} & \dots & y_k \end{bmatrix}^\top, \mathbf{q}_k \in \mathbb{R}^{l \times 1}, \quad (3)$$

where y_{ζ_k} to y_k corresponds to a fixed size data window that moves over time, containing the last l observations of time series, including the current one. This window is different from the one used in initial conditions. Using $\mathbf{q}_k$ vector, the covariance matrix is update recursively according to:

$$\mathbf{S}_k = \mathbf{S}_{k-1} + \mathbf{q}_k \mathbf{q}_k^\top, \mathbf{S}_k \in \mathbb{R}^{l \times l}. \quad (4)$$

The covariance matrix $\mathbf{S}_k$ represents all the historical information until instant k of time series in a matrix with fixed dimension.

2.1.3 | Singular value decomposition

The next step is based on the singular value decomposition of $\mathbf{S}_k$ matrix, for $k = w + 1, w + 2, \dots$. For each k , l eigenvalues and eigenvectors are computed. The eigenvalues and their associated orthonormal eigenvectors are represented, respectively, by:

$$\begin{aligned} \rho_k^1 \geq \rho_k^2 \geq \dots \geq \rho_k^l \geq 0 \quad (\text{eigenvalues}), \\ \mathbf{p}_k^1, \mathbf{p}_k^2, \dots, \mathbf{p}_k^l \in \mathbb{R}^{l \times 1} \quad (\text{eigenvectors}). \end{aligned} \quad (5)$$

The decomposition of observation y_k , using the eigenvectors and updating vector $\mathbf{q}_k$, is defined by:

$$\begin{aligned} y_k &= \theta_k^1 + \theta_k^2 + \dots + \theta_k^l, \\ \theta_k^j &= \hat{p}_k^j \mathbf{q}_k^1 \mathbf{p}_k^j, \quad j = 1, \dots, l, \end{aligned} \quad (6)$$

where θ_k^j is the decomposition term of sample y_k according to each eigenvalue in Equation (5), and $\hat{p}_k^j$ is the last element of eigenvector $\mathbf{p}_k^j$. Only this element is required to proceed with the recursive component extraction procedure.

2.1.4 | Grouping step

The grouping performs the distribution of eigenvalues index set $\{1, \dots, l\}$ of Equation (6) in n disjoint subsets of indices denoted by $I_1, I_2, \dots, I_n$. Let $I = i_h, \dots, i_j$ be one of these generic subsets



of indexes. Thus, the new term associated in this case is $\theta_k^I = \theta_k^{i_h} + \dots + \theta_k^{i_j}$. By synthesizing, this procedure is performed for the defined n subsets as follows Reference 17:

$$y_k = \theta_k^{I_1} + \theta_k^{I_2} + \dots + \theta_k^{I_n}. \quad (7)$$

where $\theta_k^{I_j}$ ($j = 1, \dots, n$) are new grouped decomposition terms from Equation (6). There are several forms of grouping step,²⁰ where the simplest and computationally efficient is considering $l = n$, that is, each term in Equation (6) is related to a grouped decomposition term in Equation (7) ($\theta_k^{I_j} = \theta_k^j$). In this way, the number of parameters to be determined by the user is reduced.

2.1.5 | Unobservable components determination

Finally, each grouped decomposition term of Equation (7) is associated with an unobservable component, that is:

$$c_k^j = \theta_k^{I_j}, \text{ for } j = 1, \dots, n, \quad (8)$$

where c_k^j is j th unobservable component in instant k . Considering Equations (7) and (8), it is possible to infer that:

$$y_k = c_k^1 + c_k^2 + \dots + c_k^n, \quad (9)$$

where the sum of these components is exactly equal to the observation of time series in instant k .

2.2 | NF-TS training

For each component extracted from time series, there is a NF-TS structure to perform the forecasting separately, therefore, there are n NF-TS. These networks are generated at instant $k = w + 1$ (after the initial conditions of recursive SSA in Section 2.1.1). They are updated in an evolving way, that is, their structures are modified in order to represent the dynamics from data. In order to perform the learning or training phase of n NF-TS, this article used an approach based on ALMMo-1 method.¹² The ALMMo-1 method has a recursive characteristic and data oriented, where its parameters and structure can be obtained directly from data using empirical fuzzy sets.²¹ The learning structure is based on regions in data space called clouds, which are similar to clusters. However, the clouds are nonparametric, free of external constraints, and they do not have well-defined form.¹² The clouds directly represent the properties of local set of observed data samples. Each NF-TS has its own clouds, and it is possible to associate for each cloud a fuzzy rule.

For each instant k and individual NF-TS, it is necessary to build a data stream that contains data of components extracted in previous instants, as follows:

$$\mathbf{d}_k^j = [c_{k-1}^j \dots c_{k-r}^j c_{k-s}^j], \mathbf{d}_k^j \in \mathbb{R}^{1 \times r+1}, j = 1, \dots, n, \quad (10)$$

where $\mathbf{d}_k^j$ represents the data stream of j th component by NF-TS j ($j = 1, \dots, n$), s represents a specific instant of time series data that has good autocorrelation with the current sample or seasonal period, and r corresponds to the number of regressors. Therefore, clouds are generated in the data space with $r + 1$ dimensions, where each NF-TS has a distinct data space. For example, a data



stream for NF-TS 1, that is, the NF-TS for the first unobservable component, with $r = 2$ and $s = 12$ can be written as:

$$\mathbf{d}_k^1 = [c_{k-1}^1 \ c_{k-2}^1 \ c_{k-12}^1], \quad \mathbf{d}_k^1 \in \mathbb{R}^{1 \times 3}. \quad (11)$$

2.2.1 | Structure updating of NF-TS

Considering the n NF-TS individually, for each new $\mathbf{d}_k^j$ that is formulated, the following parameters must be updated:

$$\begin{aligned} \mu_k^j &= \frac{k-1}{k} \mu_{k-1}^j + \frac{1}{k} \mathbf{d}_k^j, \quad \mu_k^j \in \mathbb{R}^{1 \times r}, \\ \beta_k^j &= \frac{k-1}{k} \beta_{k-1}^j + \frac{1}{k} \|\mathbf{d}_k^j\|^2, \quad j = 1, \dots, n, \end{aligned} \quad (12)$$

where μ_k^j represents the global mean of $\mathbf{d}_k^j$ and β_k^j is the average scalar product. It is necessary to compute the densities of data streams $\mathbf{d}_k^j$ and all the centers of clouds, considering the NF-TS individually. Density is a measure that indicates the spatial distribution of a specific sample of data in relation to all other data samples.¹² The density D^j of a generic sample $\mathbf{z}^j$ considering an Euclidean distance and NF-TS j is Reference 12:

$$D^j(\mathbf{z}^j) = \frac{1}{1 + \frac{\|\mathbf{z}^j - \mu_k^j\|^2}{\beta_k^j - \|\mu_k^j\|^2}}. \quad (13)$$

Using the density measure, it is possible to verify if $\mathbf{d}_k^j$ can be considered a center of a new cloud. The Condition 1 below analyzes whether a new cloud or fuzzy rule must be created:¹²

$$\text{IF } D^j(\mathbf{d}_k^j) > \max_{i=1}^{R^j} D^j(\mathbf{f}_i^j) \text{ OR } D^j(\mathbf{d}_k^j) < \min_{i=1}^{R^j} D^j(\mathbf{f}_i^j), \quad (14)$$

where $D^j(\mathbf{d}_k^j)$ and $D^j(\mathbf{f}_i^j)$ correspond to the densities (Equation (13)) of data stream and the center of cloud i ($i = 1, \dots, R^j$) in $(r+1)$ -dimensional space $\mathbf{f}_i^j$, replacing $\mathbf{z}^j$ by $\mathbf{d}_k^j$ and $\mathbf{f}_i^j$, respectively. The number of clouds or fuzzy rules of NF-TS j is denoted by R^j , so the i index is $i = 1 \dots, R^j$. If Condition 1 is true, a potential new cloud or fuzzy rule is created. Before, however, it is necessary to check if an excessive overlap exists between clouds in space when a new one is generated. To avoid the redundancy and overlap in data space, the following condition is evaluated:

$$\text{Condition 2: IF } \Delta_{i,k}^j \geq \frac{1}{1 + \phi^2}, \quad (15)$$

where ϕ is an universal parameter for all NF-TS defined by expert and $\Delta_{i,k}^j$ is the density calculated by fuzzy rule i and by NF-TS j for instant k using the following equation:¹²

$$\begin{aligned} \Delta_{i,k}^j &= \frac{1}{1 + \frac{(S_i^j)^2 \|\mathbf{d}_k^j - \mathbf{f}_i^j\|^2}{(S_i^j+1)(S_i^j \|\mathbf{d}_k^j\|^2 - \|\mathbf{d}_k^j + S_i^j \mathbf{f}_i^j\|^2)}}, \\ i &= 1, \dots, R^j, \end{aligned} \quad (16)$$

where S_i^j is the support or number of points associated with cloud i and NF-TS j , χ_i^j is the average scalar product of points associated with cloud i and NF-TS j , and $\mathbf{f}_i^j$ represents the center of cloud i



of NF-TS j . Therefore, if Condition 2 (Equation (15)) is true, there is an excessive overlap because of the potential new cloud, so, there is no need to create a new cloud and only updates the nearest cloud parameters represented by the i^* index, as follows:

$$\begin{aligned}\text{Center of cloud } i^* : \mathbf{f}_{i^*}^j &= \frac{\mathbf{f}_{i^*}^j + \mathbf{d}_k^j}{2}, \\ \text{Average scalar product of cloud } i^* : \chi_{i^*}^j &= \frac{\chi_{i^*}^j + \|\mathbf{d}_k^j\|^2}{2}, \\ \text{Support of cloud } i^* : S_{i^*}^j &= S_{i^*}^j + 1.\end{aligned}\quad (17)$$

Otherwise, if Condition 2 is false, the potential new cloud is effectively established in NF-TS structure. Thus, a new fuzzy rule is created as well as a cluster with center $\mathbf{f}_i^j$ as follows:

$$\begin{aligned}R^j &= R^j + 1, \\ \mathbf{f}_i^j &= \mathbf{d}_k^j, \text{ for } i = R^j.\end{aligned}\quad (18)$$

If Condition 1 in Equation (14) is not satisfied, it means that the point $\mathbf{d}_k^j$ must be associated with the nearest cloud denoted by index i^* . The updating of cloud parameters in this case is:

$$\begin{aligned}\text{Support of cloud } i^* : S_{i^*}^j &= S_{i^*}^j + 1, \\ \text{Center of cloud } i^* : \mathbf{f}_{i^*}^j &= \frac{S_{i^*}^j - 1}{S_{i^*}^j} \mathbf{f}_{i^*}^j + \frac{1}{S_{i^*}^j} \mathbf{d}_k^j, \\ \text{Average scalar product of cloud } i^* : \chi_{i^*}^j &= \frac{S_{i^*}^j - 1}{S_{i^*}^j} \chi_{i^*}^j + \frac{1}{S_{i^*}^j} \|\mathbf{d}_k^j\|^2.\end{aligned}\quad (19)$$

The contribution or normalized activation degree $\lambda_{i,k}^j$ of each fuzzy rule i , in global output of NF-TS j , and instant k has a value between 0 and 1 and can be computed by:

$$\lambda_{i,k}^j = \frac{\Delta_{i,k}^j}{\sum_{q=1}^{R^j} \Delta_{q,k}^j}, \quad (20)$$

where the denominator is the sum of densities of all the fuzzy rules of NF-TS j in instant k . Some fuzzy rules may lose utility over time, contributing very little to NF-TS output. To simplify the model, it is interesting to remove these clouds. Therefore, a utility measure of these clouds can be computed for each instant k as follows:

$$\begin{aligned}\eta_i^j &= \frac{1}{k - T_i^j} \Lambda_i^j, \text{ for } k \neq T_i^j, \\ \eta_i^j &= 1, \text{ for } k = T_i^j,\end{aligned}\quad (21)$$

where η_i^j is the clouds utility of NF-TS j , T_i^j is the instant k that the cloud i was generated, and Λ_i^j brings the sum of all activation degrees $\lambda_{i,k}^j$ of that fuzzy rule over all the instants k since its inception. The condition for removing rules is:

$$\text{Condition 3: IF } \eta_i^j < \eta_0 \text{ THEN } R^j = R^j - 1, \quad (22)$$



where η_0 is a small tolerance constant. The clouds that satisfies the Condition 3 above are removed, simplifying the structure of each NF-TS.

2.2.2 | Estimation of consequent parameters

In this step, the consequent parameters of NF-TS are updated. The structure of a fuzzy rule is represented for each NF-TS $j = 1, \dots, n$ by:

$$\begin{aligned} \text{IF } (c_{k-1}^j \sim f_i^{j,1}) \text{ AND } (c_{k-2}^j \sim f_i^{j,2}) \dots \text{ AND } (c_{k-s}^j \sim f_i^{j,r+1}) \\ \text{THEN } \hat{c}_{i,k}^j = \sum_{q=1}^r \alpha_i^{j,q} c_{k-q}^j + \alpha_i^{j,r+1} c_{k-s}^j + b_i^j \end{aligned} \quad (23)$$

where $c_{k-1}^j, c_{k-2}^j, \dots, c_{k-s}^j$ are j th unobservable component data; $f_i^{j,1}, f_i^{j,2}, \dots, f_i^{j,r}$ are the centers of clouds for each dimension in $(r+1)$ -dimensional data space, considering each fuzzy rule $i = 1, \dots, R^j$; $\hat{c}_{i,k}^j$ represents the local forecasting of fuzzy rule i and NF-TS j ; $\alpha_i^{j,1}, \alpha_i^{j,2}, \dots, \alpha_i^{j,r+1}$ are parameters of consequents that must be estimated for fuzzy rule i and NF-TS j ; and b_i^j is a bias of equation of fuzzy rule i and NF-TS j .

In order to estimate these consequents parameters, the fuzzy weighted recursive least squares method²² is used for each fuzzy rule, considering the normalized activation degree $\lambda_{i,k}^j$ as a weighting factor as follows:

$$\begin{aligned} G_{i,k}^j &= \frac{P_{i,k-1}^j v^T}{v \lambda_{i,k}^j P_{i,k-1}^j v^T + 1}, \quad G_{i,k}^j \in \mathbb{R}^{(r+1) \times 1}; \\ \alpha_{i,k}^j &= \alpha_{i,k-1}^j + \lambda_{i,k}^j G_{i,k}^j (c_k^j - v \alpha_{i,k-1}^j), \quad \alpha_{i,k}^j \in \mathbb{R}^{(r+1) \times 1}; \\ P_{i,k}^j &= P_{i,k-1}^j - \lambda_{i,k}^j G_{i,k}^j v P_{i,k-1}^{jT}, \quad P_{i,k}^j \in \mathbb{R}^{(r+1) \times (r+1)}; \\ \text{for } v &= [d_k^j \ 1] \in \mathbb{R}^{1 \times (r+1)}. \end{aligned} \quad (24)$$

where $G_{i,k}^j$ is the gain matrix for each fuzzy rule and NF-TS j , $P_{i,k}^j$ is the i th fuzzy rule covariance matrix from NF-TS j , $\alpha_{i,k}^j$ is the parameters vector, and v has regressors data to estimate c_k^j .

2.2.3 | Outputs of NF-TS

From the outputs of local submodels $\hat{c}_{i,k}^j$ represented by fuzzy rules, it is possible to obtain the global unobservable component forecasting $\hat{c}_k^j$ as follows:

$$\hat{c}_k^j = \sum_{i=1}^{R^j} \lambda_{i,k}^j \hat{c}_{i,k}^j. \quad (25)$$

Adding the forecasting results of each component using the distributed NF-TS (Equation (25)), the forecasting of original time series $\tilde{y}_k$ is obtained by:

$$\tilde{y}_k = \sum_{j=1}^n \hat{c}_k^j. \quad (26)$$



Algorithm 1 shows the training procedure for proposed method.

Algorithm 1. Training Step of Proposed Method

```

1: Set  $w$ ,  $l$ , and  $n$  (initial conditions of recursive SSA);
2: Set  $r$ ,  $\eta_0$ , and  $\phi$ ;
3:  $\zeta_w = w - l + 1$ ;
4: Trajectory matrix  $\mathbf{M}$  (Equation (1));
5: Initial covariance matrix (Equation (2));
6: while there is time series sample  $y_k$  do
7:   Updating vector (Equation (3));
8:   Recursive updating of covariance matrix (Equation (4));
9:   Singular value decomposition (Equation (6));
10:  Grouping step (Equation (7));
11:   $n$  unobservable components (Equation (8)).
12:  ***** Distributed Procedure (a NF-TS for each component  $j$ );
13:  for  $j = 1, \dots, n$  do
14:    Data stream formulation  $\mathbf{d}_k^j$  (Equation (10));
15:    if  $k = w + 1$  then
16:      Initialization of first cloud;
17:    else
18:      Update  $\mu_k^j$  and  $\rho_k^j$  (Equation (12));
19:      Computing  $\mathbf{d}_k^j$  densities and focal points densities of each cloud  $i$ , for  $i = 1, \dots, R^j$ 
      (Equation (13));
20:      if Condition 1 is true (Equation (14)) then
21:        if Condition 2 is true (Equation (15)) then
22:          Overlapping clouds (Equations (16) and (17));
23:        else
24:          New cloud ( $R^j = R^j + 1$ );
25:        end if
26:      else
27:        Associate the data point to the nearest cloud (Equation (19));
28:      end if
29:      Computing normalized activation degree (Equation (20));
30:      if Condition 3 is true (Equation (22)) then
31:        Remove fuzzy rules;
32:      end if
33:      Recursive parameters estimation (Equation (24));
34:      Fuzzy inference  $\tilde{z}_k^j$  (Equations (23) and (25));
35:    end if
36:  end for
37:  Time series forecasting  $\hat{y}_k$  (Equation (26)).
38: end while
  
```



2.3 | Forecasting step

After training process, the proposed method perform the recursive multistep forecasting according to forecasting horizon parameter h . When the forecasting is made, there is not available sample y_k from time series, therefore, it is not possible to extract new unobservable components. Because the unobservable components data are fundamental in formulation of fuzzy rules (Equation (23)), the components forecasting results are feedback into the respective NF-TS j in order to the fuzzy inference can be performed. In the forecasting or test step, the NF-TS networks do not evolve, that is, they do not modify their structure, remaining fixed after training. Algorithm 2 shows the forecasting procedure of proposed methodology.

Algorithm 2. Forecasting or Test Step of Proposed Method

```

1: while the forecasting horizon  $h$  is not reached do
2:   ***** Distributed Procedure (a NF-TS for each component  $j$ );
3:   for  $j = 1, \dots, n$  do
4:     Feedback from previous components forecasts to formulate  $\mathbf{d}_k^j$ ;
5:     Compute the centers densities of each cloud (Equation (13));
6:     Computing normalized activation degree (Equation (20));
7:     Fuzzy inference  $\tilde{c}_k^j$  (Equations (23) and (25));
8:   end for
9:   Time series forecasting  $\tilde{y}_k$  (Equation (26)).
10: end while

```

3 | EXPERIMENTAL RESULTS

3.1 | Experiment 1

In this section, the proposed methodology is evaluated using a recursive multistep ahead forecasting for univariate time series, comparing its results with Nguyen and Novak.¹¹ These authors also consider a similar procedure, performing the components forecasting separately and then grouping the individual results to forecast the original time series. The classic ARIMA method, that includes difference operators, autoregressive terms, moving average terms, seasonal difference operators, seasonal autoregressive terms, and seasonal moving average term, is also considered. The other method used for comparison is seasonal trend decomposition based on loess, that is, a forecasting technique based on the decomposition model of time series. Thus, the following time series are considered in this experiment:¹¹

1. TS1: Monthly number of unemployed persons in Australia (September 1978-August 1995). No missing values and without outliers. Total samples: $N = 211$;
2. TS2: Quarterly gross fixed capital expenditure—private nondwelling construction, Australia: millions of dollars, 1984/1985 prices. Q3 1960-Q2 1991. No missing values and without outliers. Total samples: $N = 127$;
3. TS3: Mean monthly flow, piper's hole River, January 1953-December 1981. No missing values and without outliers. Total samples: $N = 348$;
4. TS4: Quarterly production of clay bricks: million units. Q4 1956-Q3 1994. Total samples: $N = 155$;

- 5. N2090: time series from the M3 Competition.²³ No missing values and without outliers. Total samples: $N = 144$;
- 6. N2076: time series from the M3 Competition.²³ No missing values and without outliers. Total samples: $N = 72$;
- 7. N912: time series from the M3 Competition.²³ No missing values and without outliers. Total samples: $N = 144$;
- 8. N897: time series from the M3 Competition.²³ No missing values and without outliers. Total samples: $N = 72$.

Table 1 shows the parameters required to perform the experiment of each time series using the approach proposed in this article. The parameters with the best results were selected. The forecasting horizon h is the same considered in Nguyen and Novak.¹¹ In defining the parameters, the following aspects were considered:

- the initial window is w , for $w > s$ to possibility c_{k-s}^j term in data stream;
- n unobservable components for each time series, where $l = n$ (Section 2.1.4);
- s is the seasonal period of time series;
- h is the forecasting horizon;
- r is the number of regressors in data stream (Equation (10));
- ϕ is the parameter of Equation (15). According to the experiments, this parameter does not have much influence on forecasting results in the range between $0.1 < \phi < 0.3$. Outside this range, it can negatively influence in NF-TS results;
- η_0 is the parameter of Equation (22), for $0 < \eta_0 < 0.5$. Outside this range, it can negatively influence in NF-TS results.

For example, two NF-TS were generated for time series N2090, one for each component ($n = 2$). So, a fuzzy rule structure for both NF-TS, where $r = 1$ and $s = 12$, is represented by:

NF-TS 1: IF $(c_{k-1}^1 \sim f_i^{1,1})$ AND $(c_{k-12}^1 \sim f_i^{1,2})$
THEN $\hat{c}_{i,k}^1 = a_i^{1,1}c_{k-1}^1 + a_i^{1,2}c_{k-12}^1 + a_i^{1,3}$

NF-TS 2: IF $(c_{k-1}^2 \sim f_i^{2,1})$ AND $(c_{k-12}^2 \sim f_i^{2,2})$
THEN $\hat{c}_{i,k}^2 = a_i^{2,1}c_{k-1}^2 + a_i^{2,2}c_{k-12}^2 + a_i^{2,3}$.

(27)

TABLE 1 Parameters of proposed methodology for each time series in Experiment 1

Time series	w	$l = n$	s	h	r	ϕ	η_0
TS1	18	9	12	20	2	0.3	0.3
TS2	9	4	6	12	5	0.3	0.25
TS3	29	8	23	18	4	0.3	0.4
TS4	12	5	9	16	5	0.3	0.35
N2090	13	2	12	18	1	0.3	0.1
N2076	15	5	6	8	5	0.3	0.2
N912	18	5	12	18	1	0.3	0.05
N897	13	4	8	8	5	0.3	0.25

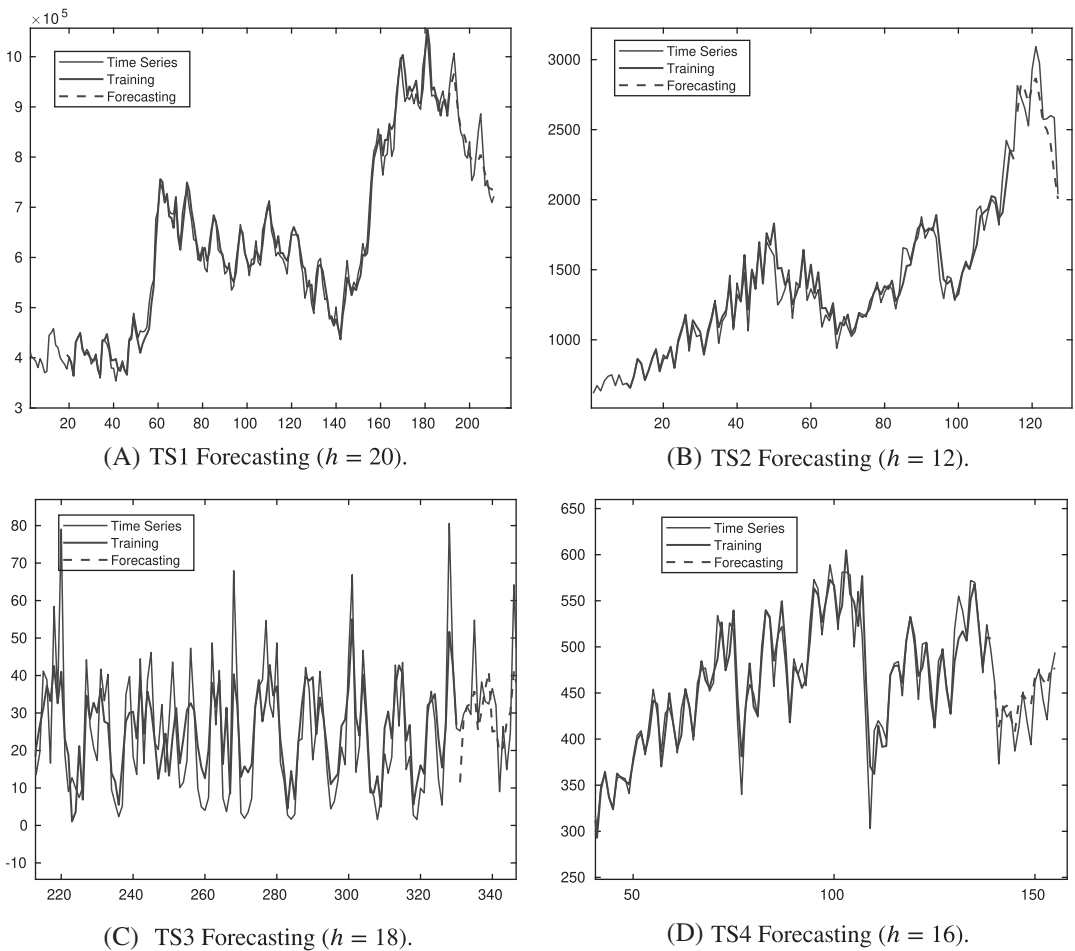


FIGURE 2 Training and forecasting of proposed methodology in Experiment 1 (Part 1)

Figures 2A-D and 3A-D show the training and forecasting results of eight time series considered according of parameters in Table 1. The proposed distributed methodology in this article based on evolving NF-TS presents good approximations in training and forecasting steps. Some figures have been approximated for better viewing but note that the initial part before the training step corresponds to the initial window w commented in Section 2.1.1.

Besides that, Figure 4A-D show, in more detail, the components and fuzzy rules in the case of time series N2090, displaying the forecasting results of $n = 2$ NF-TS. Each NF-TS tries to forecast a separate unobservable component in training step, adding or removing fuzzy rules or clouds. In forecasting step, the number of clouds or fuzzy rules does not change, showing that the NF-TS structure is fixed in this step.

Table 2 shows the comparative results of proposed methodology with three approaches presented in Nguyen and Novak,¹¹ considering the parameters of Table 1. It can be seen that the presented strategy obtained competitive results according to RMSE and MAPE criteria during the

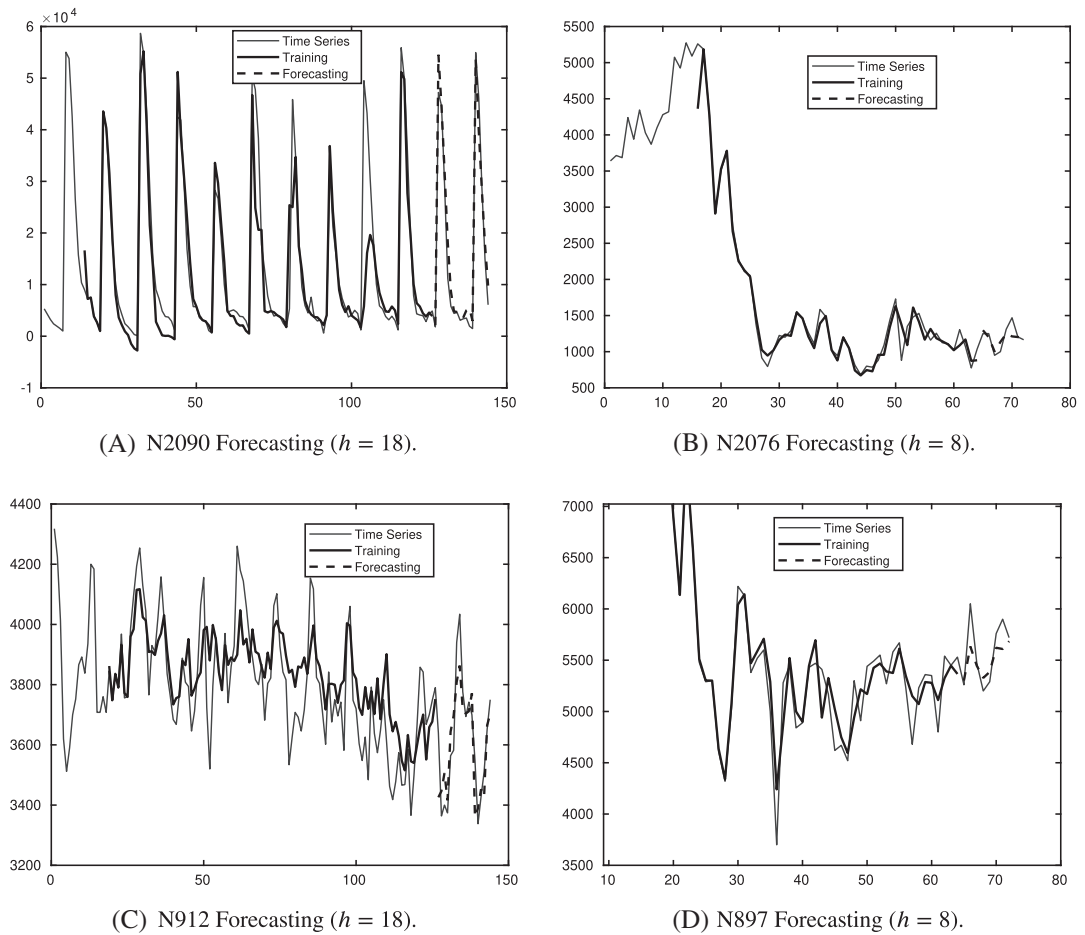


FIGURE 3 Training and forecasting of proposed methodology in Experiment 1 (Part 2)

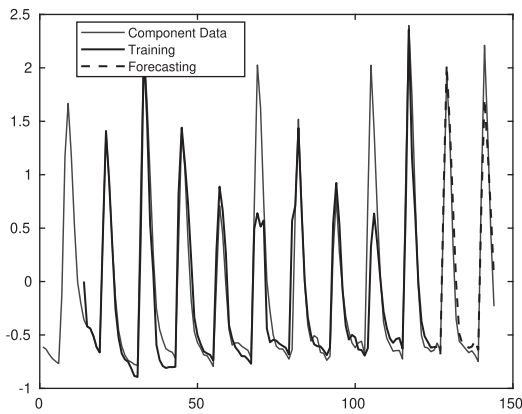
forecasting horizon h , which are defined mathematically by:

$$\text{RMSE} = \sqrt{\frac{1}{h} \sum_{t=1}^h (y_t - \tilde{y}_t)^2},$$

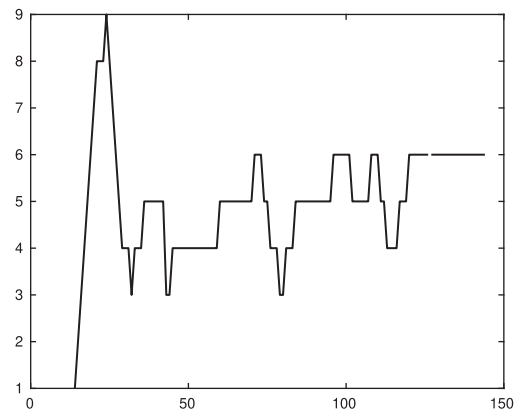
$$\text{MAPE} = \frac{100\%}{h} \sum_{t=1}^h \left| \frac{y_t - \tilde{y}_t}{y_t} \right|. \quad (28)$$

The proposed method was better in all cases according with these criteria, except for the N912 time series, where Nguyen and Novak method¹¹ was superior. In the case of TS3 series, the distributed NF-TS were better considering the RMSE and practically the same result for MAPE.

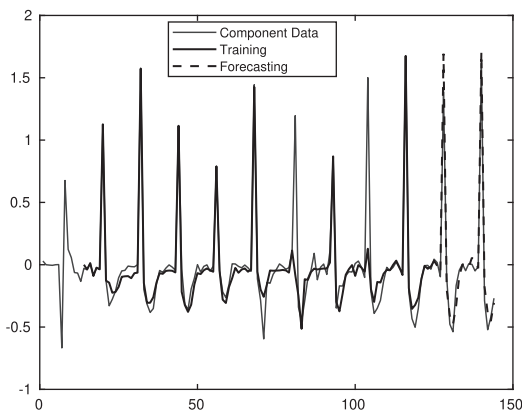
Finally, in Table 3, the number of clouds or fuzzy rules generated by each NF-TS and time series in Experiment 1 are showed. The gaps in the table indicate that this amount of NF-TS or components was not generated for specific time series. Each NF-TS has a number of fuzzy rules that depend on the behavior of unobservable component related. Considering the number of fuzzy



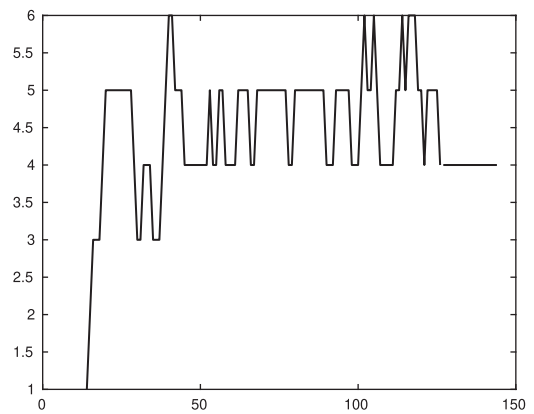
(A) Component 1 of N2090 (NF-TS 1).



(B) Evolution of Clouds Number of NF-TS 1.



(C) Component 2 of N2090 (NF-TS 2).



(D) Evolution of Clouds Number of NF-TS 2.

FIGURE 4 Unobservable components forecasting and evolution of fuzzy rules or clouds number of time series N2090

rules for each case, it is noticed that each NF-TS adjusts differently according to the dynamics of the related unobservable component.

3.2 | Experiment 2

In this experiment, the proposed methodology is compared with RBF-AR model²⁴ and ANN + DSDT.²⁵ Both methods results in forecasting or test step are presented in Gan et al²⁴ article. The radial basis function network-based state-dependent autoregressive (RBF-AR) model is derived by using a set of RBF networks to approximate the coefficients of a state-dependent AR model. Gan et al²⁴ used a RBF-AR and combined genetic optimization and gradient-based optimization algorithm to forecasting the seasonal and trend time series. On the other hand, the ANN + DSDT method²⁵ is an ANN with detrended and deseasonalize used to univariate time series forecasting as well.

The following real time series from US retail sales are considered:²⁴

TABLE 2 Comparative results between the proposed methodology and methods presented in Nguyen and Novak¹¹ according to RMSE and MAPE criteria (best in **bold**)

Time series	Proposed method		Nguyen and Novak ¹¹		STL		ARIMA	
	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE
TS1	32 329.26	0.031	47 807	0.05	58 662	0.07	89 893	0.09
TS2	187.61	0.064	274.01	0.08	274.39	0.08	337.07	0.11
TS3	9.85	0.23	10.29	0.23	16.42	0.41	15.81	0.38
TS4	21.78	0.041	23.35	0.04	29.46	0.05	81.85	0.18
N2090	2832.19	0.269	4309	0.326	6544	0.396	4431.5	0.635
N2076	110.08	0.064	218.59	0.172	158.29	0.097	205,81	0.128
N912	102.54	0.023	65.48	0.015	206.18	0.047	171.35	0.039
N897	195.67	0.025	199.17	0.031	248.58	0.033	305.69	0.042

Abbreviations: ARIMA, autoregressive integrated moving average; STL, seasonal trend decomposition based on loess.

TABLE 3 Number of clouds or fuzzy rules for each distributed NF-TS in Experiment 1

Time series	NF-TS 1	NF-TS 2	NF-TS 3	NF-TS 4	NF-TS 5	NF-TS 6	NF-TS 7	NF-TS 8	NF-TS 9
TS1	3	4	2	4	4	3	3	3	2
TS2	2	2	2	4	–	–	–	–	–
TS3	4	3	4	2	3	3	2	3	–
TS4	3	2	4	4	2	–	–	–	–
N2090	6	4	–	–	–	–	–	–	–
N2076	2	6	3	5	4	–	–	–	–
N912	5	6	3	4	4	–	–	–	–
N897	4	2	4	2	–	–	–	–	–

1. Book Stores from US census Bureau.²⁴ No missing values and without outliers. Total samples: $N = 120$;
2. Clothing Stores from US census Bureau.²⁴ No missing values and without outliers. Total samples: $N = 120$;
3. Furniture Stores from US census Bureau.²⁴ No missing values and without outliers. Total samples: $N = 120$;
4. Hardware Stores from US census Bureau.²⁴ No missing values and without outliers. Total samples: $N = 120$.

All these time series starts from January 1992 and ends in December 2001, they are monthly with seasonal period $s = 12$, there are 120 data samples, and the forecasting horizon is $h = 12$. Similar to Experiment 1 considerations, the parameters selected in this case are represented in Table 4. For simplicity in choosing the ϕ parameter, note that its value is the same as in Experiment 1 ($\phi = 0.3$), because in the range $[0.1; 0.3]$ it does not influence in forecasting results. So, the user

Time series	w	$l = n$	s	h	r	ϕ	η_0
Book	15	7	12	12	1	0.3	0.2
Clothing	29	7	12	12	1	0.3	0.1
Furniture	29	5	12	12	1	0.3	0.01
Hardware	26	10	12	12	1	0.3	0.15

TABLE 4 Experiments parameters of proposed methodology for each time series in Experiment 2

TABLE 5 Comparative results in training and test steps between the proposed methodology and methods RBF-AR²⁴ and ANN + DSDT²⁵ according to RMSE (best in **bold**)

Time series	Proposed method		RBF-AR ²⁴		ANN + DSDT ²⁵	
	RMSE train	RMSE test	RMSE train	RMSE test	RMSE train	RMSE test
Book	32.88	35.99	62.63	96.80	37.39	88.74
Clothing	310.06	300.79	244.89	337.72	174.07	315.43
Furniture	73.28	156.25	77.85	191.54	52.53	99.45
Hardware	48.98	22.07	40.31	23.53	26.16	49.17

Abbreviations: ANN, artificial neural network; RBF-AR, radial basis function network-based state-dependent autoregressive.

does not need to set a specific value for this parameter. Another important characteristic indicated by Table 4 is about the parameter r , whose value was $r = 1$ in all cases. Thus, the distributed RNF-TS receive a very simple data stream, as in:

$$\mathbf{d}_k^j = [c_{k-1}^j \ c_{k-12}^j], \mathbf{d}_k^j \in \mathbb{R}^{1 \times 2}. \quad (29)$$

The RBF-AR method,²⁴ for example, requires samples from the series at different time points in the range from $t - 1$ to $t - 12$ according to the analyzed time series. Therefore, an advantage of the distributed RNF-TS is its simplicity considering this experiment, requiring only the information on the $t - 1$ and $t - 12$ to perform the forecasting.

The comparison in training and forecasting (test) steps according wit RMSE criteria is presented in Table 5. The RBF-AR and ANN+DSDT methods applied a preprocessing in time series before computing the forecasting. The RBF-AR applied the differencing approach in data to obtain a stationary time series. On the other hand, ANN+DSDT removes the trend and seasonality previously. However, the proposed approach it does not perform this batch preprocessing, considering the original nonstationary data and the unobservable components to perform the forecasting and learning incrementally, that is, sample by sample from time series.

Table 5 shows that the results of proposed approach are competitive considering these time series and state-of-the-art methods. The RMSE in test or forecasting step was lower in three cases for the proposed method. However, it was lower only for Book time series in training step. It is important to avoid models with overfitting characteristics, because they present small errors in training, but, in the validation or test steps, they do not achieve good results. The proposed approach does not have this unwanted characteristic according with Table 5.

Finally, Table 6 show the RMSE in forecasting step ($h = 12$) for different number of unobservable components, $2 \leq n \leq 10$. For Book time series, in all cases the proposed approach was the best in comparison with RBF-AR and ANN + DSDT approaches. For Clothing and Hardware

TABLE 6 Best RMSE results in forecasting step of proposed method for $2 \leq n \leq 10$

Number of components n	Book	Clothing	Furniture	Hardware
2	46.57	480.28	212.01	33.92
3	43.59	357.80	185.06	33.30
4	52.35	432.42	180.05	26.50
5	43.04	360.85	156.25	29.30
6	50.30	349.88	172.18	32.27
7	35.99	300.79	162.20	30.36
8	48.09	479.95	174.21	27.72
9	42.25	415.30	167.08	25.34
10	42.30	350.50	161.57	22.07

Note: In **bold**, the values lower than RBF-AR and ANN + DSDT approaches results.
Abbreviations: ANN, artificial neural network; RBF-AR, radial basis function network-based state-dependent autoregressive.

time series, the method was superior in only one case in bold. In the case of Furniture series, the proposed approach was not the best in any case. This shows that the determination of the unobservable components number n depends on the characteristic of each time series, and it is an important parameter for proposed approach to present the best results.

4 | CONCLUSIONS

In this article, an evolving methodology based on distributed NF-TS networks is presented. The originality of this article consists in the use of evolving NF-TS to forecast each unobservable component extracted from original time series. In this case, each component has an associated evolving NF-TS. The forecasting of each component is grouped to obtain the time series future observations. This article highlights the idea of divide a complex problem into several easier subproblems using a hybrid method with NF-TS and a version of SSA method. The results of the distributed methodology showed competitive error rates compared with other state-of-art methods, highlighting the potential of proposed approach.

In future works, the following objectives will be explored:

- Define a criterion to set the parameters of proposed methodology;
- Reduce the number of parameters, developing a more autonomous and data-driven approach;
- Development a generic version of proposed methodology for multivariate time series;
- Compare methods to highlight the computational efficiency of the proposed method.

ACKNOWLEDGMENTS

The authors would like to thank the Master’s and Ph.D. Program in Electrical Engineering (PPGEE-UFMA) and Laboratory of Computational Intelligence Applied to Technology (ICAT-IFMA) for encouraging and development of this research.

ORCID

Selmo Eduardo Rodrigues Júnior  <https://orcid.org/0000-0001-8920-4023>

REFERENCES

1. De Gooijer J, Hyndman R. 25 years of time series forecasting. *Int J Forecast*. 2006;22:443-473.
2. Sagheer A, Kotb M. Time series forecasting of petroleum production using deep LSTM recurrent networks. *Neurocomputing*. 2019;323:203-213.
3. Tealab A. Time series forecasting using artificial neural networks methodologies: a systematic review. *Future Comput Inform J*. 2018;3:334-340.
4. Wang J, Wang J, Fang W, Niu H. Financial time series prediction using elman recurrent random neural networks. *Comput Intell Neurosci*. 2016;12:1-14. <http://dx.doi.org/10.1155/2016/4742515>.
5. Wang L, Zou H, Su J, Li L, Chaudhry S. An ARIMA-ANN hybrid model for time series forecasting. *Syst Res Behav Sci*. 2013;30:244-259.
6. Yolcu U, Egrioglu E, Aladag C. A new linear & nonlinear artificial neural network model for time series forecasting. *Decis Support Syst*. 2013;54:1340-1347.
7. Tealab A, Hefny H, Badr A. Forecasting of nonlinear time series using artificial neural network. *Future Comput Inform*. 2018;3:143-151.
8. Ni L, Li Y, Wang X, Zhang J, Yu J, Qi C. Forecasting of Forex time series data based on deep learning. *Proc Comput Sci*. 2019;147:647-652.
9. Yang H, Chen Y. Hybrid deep learning and empirical mode decomposition model for time series applications. *Expert Syst Appl*. 2019;120:128-138.
10. Joo T, Kim S. Time series forecasting based on wavelet filtering. *Expert Syst Appl*. 2015;42:3868-3874.
11. Nguyen L, Novak V. Forecasting seasonal time series based on fuzzy techniques. *Fuzzy Sets Syst*. 2019;361:114-129.
12. Angelov P, Gu X, Príncipe J. Autonomous learning multi-model systems from data streams. *IEEE Trans Fuzzy Syst*. 2018;26:2213-2224.
13. Lughofer E, Sayed-Mouchaweh M. Autonomous data stream clustering implementing split-and-merge concepts Towards a plug-and-play approach. *Inf Sci*. 2018;304:54-79.
14. Lughofer E. On-line assurance of interpretability criteria in evolving fuzzy system - achievements, new concepts and open issues. *Inf Sci*. 2013;251:22-46.
15. Rocha O, Serra G. Adaptive neuro-fuzzy black-box modeling based on instrumental variable evolving algorithm. *J Control Autom Syst*. 2017;28:50-67.
16. Novák V, Perfilieva I, Dvorák A. *Insight into Fuzzy Modeling*. Hoboken, NJ: John Wiley & Sons; 2016.
17. Abdollahzade M, Miranian A, Hassani H, Iranmanesh H. A new hybrid enhanced local linear neuro-fuzzy model based on the optimized singular spectrum analysis and its application for nonlinear and chaotic time series forecasting. *Inf Sci*. 2015;361:107-125.
18. Wu L, Liu S, Yang Y. Grey double exponential smoothing model and its application on pig price forecasting in China. *Appl Soft Comput*. 2016;391:117-123.
19. Bergmeir C, Hyndman R, Bentez J. Bagging exponential smoothing methods using STL decomposition and Box-Cox transformation. *Int J Forecast*. 2016;32:303-312.
20. Golyandina N, Nekrutkin V, Zhigljavsky A. *Analysis of Time Series Structure: SSA and Related Techniques*. New York, NY/London, UK: Chapman & Hall/CRC; 2001.
21. Angelov P, Gu X. Empirical Fuzzy Sets. *Int J Intell Syst*. 2017;33:362-395.
22. Angelov P. Evolving Takagi-Sugeno fuzzy systems from streaming data. In: Angelov P, Filev D, Kasabov N, eds. *Evolving Intelligent Systems: Methodology and Applications*. IEEE Press Series on Computational Intelligence. Hoboken, NJ: John Wiley & Sons; 2010.
23. Makridakis S, Hibon M. The M3-Competition: results, conclusions and implications. *Int J Forecast*. 2000;16:451-476.



24. Gan M, Cheng Y, Liu K, Zhang G. Seasonal and trend time series forecasting based on a Quasi-linear autoregressive model. *Appl Soft Comput*. 2014;24:13-18.
25. Zhang G, Qi M. Neural network forecasting for seasonal and trend time series. *Eur J Oper Res*. 2005;160:501-514.

How to cite this article: Rodrigues Júnior SE, de Oliveira Serra GL. Intelligent forecasting of time series based on evolving distributed Neuro-Fuzzy network. *Computational Intelligence*. 2020;36:1394–1413. <https://doi.org/10.1111/coin.12383>